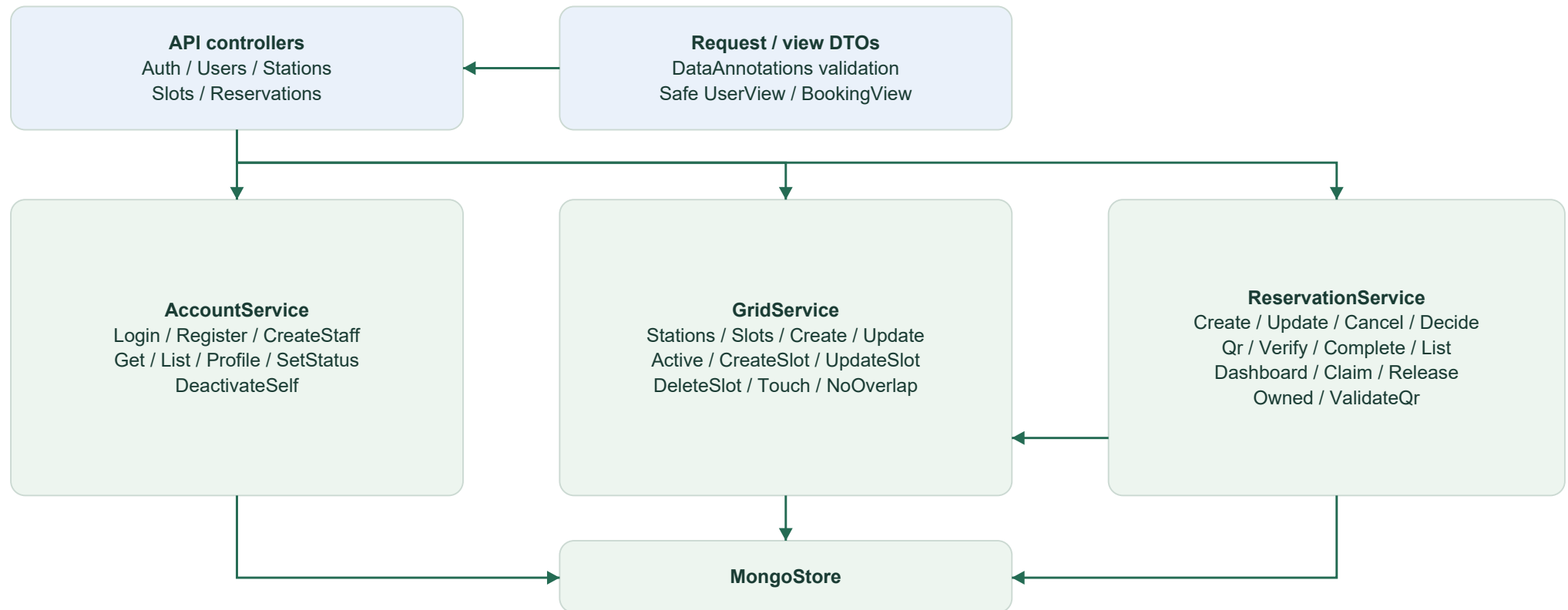


Low-level component design

04 / Classes, storage schemas, APIs and transaction algorithms

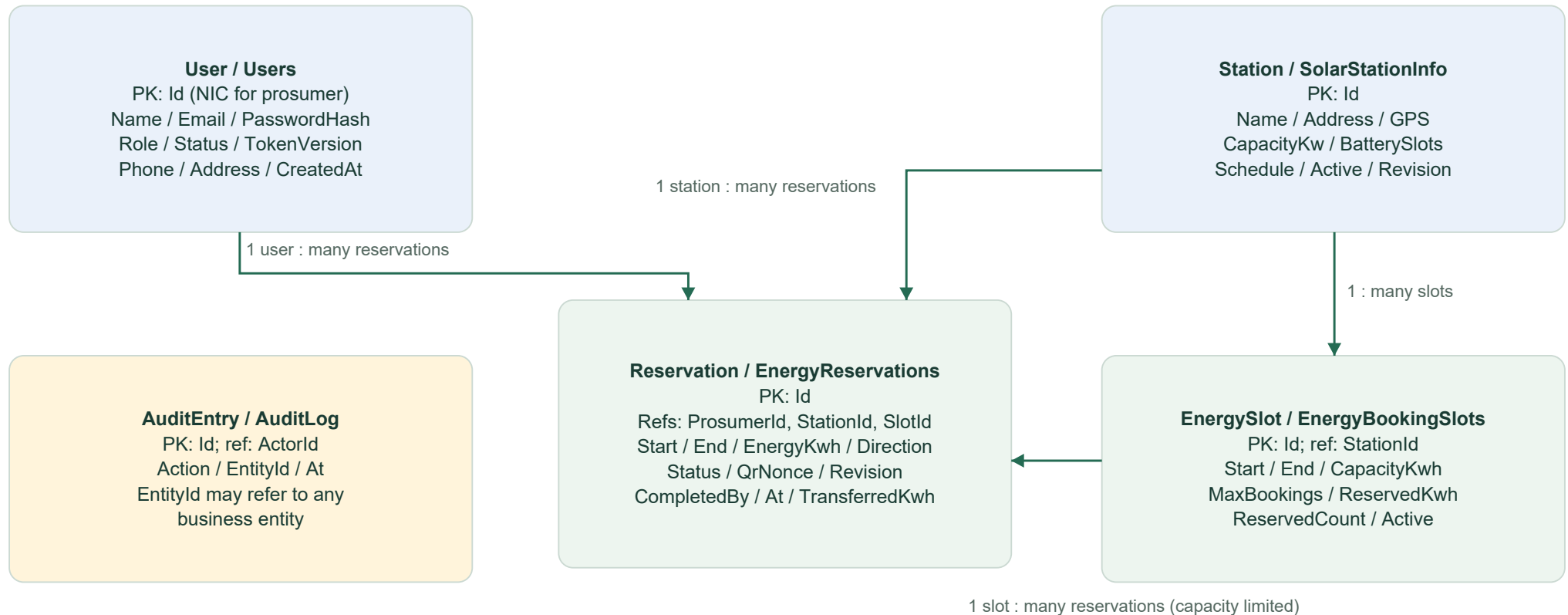


DI lifetimes: MongoStore and TimeProvider are singleton; AccountService, GridService and ReservationService are scoped; startup and expiry tasks are hosted services. BusinessRules exposes static domain checks.

Controllers share ApiController.Actor and Role from JWT claims. DTOs bind JSON and validate contracts; UserView and BookingView omit password hashes and QR nonces.

MongoDB entity relationships

References are string IDs managed by application logic; MongoDB does not enforce relational foreign keys.



User.Id is normalized NIC for prosumers and a 32-character generated GUID string for staff. Station, slot, reservation and audit identifiers also use GUID strings without hyphens. CompletedBy references a staff user.

Schema: accounts, stations and slots

Field names are C# properties; JSON responses use camelCase.

Entity / collection	Fields and types	Rules / indexing
User / Users	Id:string; Nic:string?; Name, Email, Phone, Address, PasswordHash, Role, Status:string; TokenVersion:int; CreatedAt:DateTime	Unique Email index; normalized lowercase email. Prosumer Id/Nic uppercase. Status Pending / Active / Deactivated. Role Prosumer / Backoffice / GridOperator.
Station / SolarStationInfo	Id, Name, Address, Schedule:string; Latitude, Longitude, CapacityKw:double; BatterySlots, Revision:int; Active:bool	Latitude -90..90; longitude -180..180; capacity 0.01..1,000,000; batteries 1..10,000. Active defaults true. Revision increments to conflict with simultaneous operations.
EnergySlot / EnergyBookingSlots	Id, StationId:string; Start, End:DateTime; CapacityKwh, ReservedKwh:double; MaxBookings, ReservedCount:int; Active:bool	Index StationId + Start. Capacity 0.01..1,000,000; count 1..10,000 and not above BatterySlots; no active interval overlap at a station.

What the database is not storing

There is no separate charger, vehicle, meter telemetry, tariff, invoice, payment, notification or customer-consent entity. Node Schedule is a text description; slot timestamps enforce actual booking windows.

Schema: reservations, audit and local cache

Reservation times are copied from the selected slot to preserve the booking contract.

Entity / store	Fields / relationships	Behavior
Reservation / EnergyReservations	Id, ProsumerId, StationId, SlotId:string; Start/End/CreatedAt/UpdatedAt:DateTime; EnergyKwh:double; Direction/Status/QrNonce:string; Revision:int; CompletedBy:string?; CompletedAt:DateTime?; TransferredKwh:double?	Indexes: ProsumerId + Status + Start; StationId + Status. Terminal history remains. Current QR nonce cleared when invalidated.
AuditEntry / AuditLog	Id, ActorId, Action, EntityId:string; At:DateTime	Operational actor/action/entity log. Expiry uses actor system. Many mutations audit transactionally; coverage is not universal.
Android / solara.db	cache(name TEXT PRIMARY KEY, value TEXT NOT NULL, updated_at INTEGER NOT NULL)	Session is encrypted JSON encoded as Base64 IV:ciphertext. Station reference caches are plain JSON values. Sign-out clears cache rows.
Web / sessionStorage	solara.session JSON: token + safe user + expiry	Per-tab session persistence. Logout removes key and resets in-memory data. No server-side browser session table.

Android key alias: solara.session. AES/GCM/NoPadding with a non-exportable Android Keystore secret key; unreadable session data is discarded. SQLite is a cache, not an offline transaction authority.

API contracts: identity and grid supply

All paths are relative to /api. P = active Prosumer; A = Backoffice; O = GridOperator; S = A or O.

Method / route	Role	Request / operation
POST /auth/register; POST /auth/login	Public	RegisterRequest (NIC/profile/password); LoginRequest (email/password). Register creates Pending; login requires Active.
GET, PUT /auth/me; POST /auth/me/deactivate	Any; P for deactivate	Own UserView / ProfileRequest (name, phone, address). Self-deactivation revokes sessions.
GET /users; POST /users/staff; POST /users/prosumers	S; A for create	Filtered safe accounts; O sees prosumers only. StaffRequest defines role; RegisterRequest creates active prosumer.
PUT /users/{id}; POST /users/{id}/activate or deactivate	A	Edit prosumer profile; change account state and increment token version. Self-administration via this lifecycle endpoint is blocked.
GET /stations; POST /stations; PUT /stations/{id}	Any; A create; S edit	StationRequest: name,address,latitude,longitude,capacityKw,batterySlots,schedule. GET optionally supports radius; mobile no longer supplies it.
POST /stations/{id}/activate or deactivate; DELETE /stations/{id}	A	Soft lifecycle, history retained; deactivation blocked by Pending/Approved reservations.
GET /slots; POST /stations/{id}/slots; PUT, DELETE /slots/{id}	Any list; S mutate	SlotRequest: start/end with offset, capacityKwh, maxBookings. Reserved slots cannot edit/archive.

API contracts: reservations and QR

Creation returns Pending; approval is a separate staff action.

Method / route	Role	Contract / result
GET /reservations	Any	status/search/from/to/page/pageSize; own records for P. Result items,total,page,pageSize. Page size clamped 1..100.
GET /reservations/dashboard	Any	pending, approvedFuture, completed, activeNodes. Reservation counts scoped to owner for P.
POST /reservations	Any	slotId,energyKwh,direction (DropOff Charging), optional prosumerId. P target is always actor; staff must specify active prosumer.
PUT /reservations/{id}; POST /{id}/cancel	Owner or S	Update uses ReservationRequest; status returns Pending and QR is invalidated. Cancel requires 12h notice.
POST /reservations/{id}/approve or reject	S	Pending-only decision before start; approval issues nonce; rejection releases capacity.
GET /reservations/{id}/qr	Owner or S	qrCode,reservationId,expiresAt; valid Approved booking only. Android renders QR bitmap.
POST /reservations/verify; POST /reservations/complete	S	QrRequest {qrCode}; CompleteRequest {qrCode,transferredKwh}. Returns safe BookingView.
GET /health	Public	Mongo ping; healthy/database/utc fields. A database error is reported through error middleware.

Create and modify: transaction algorithms

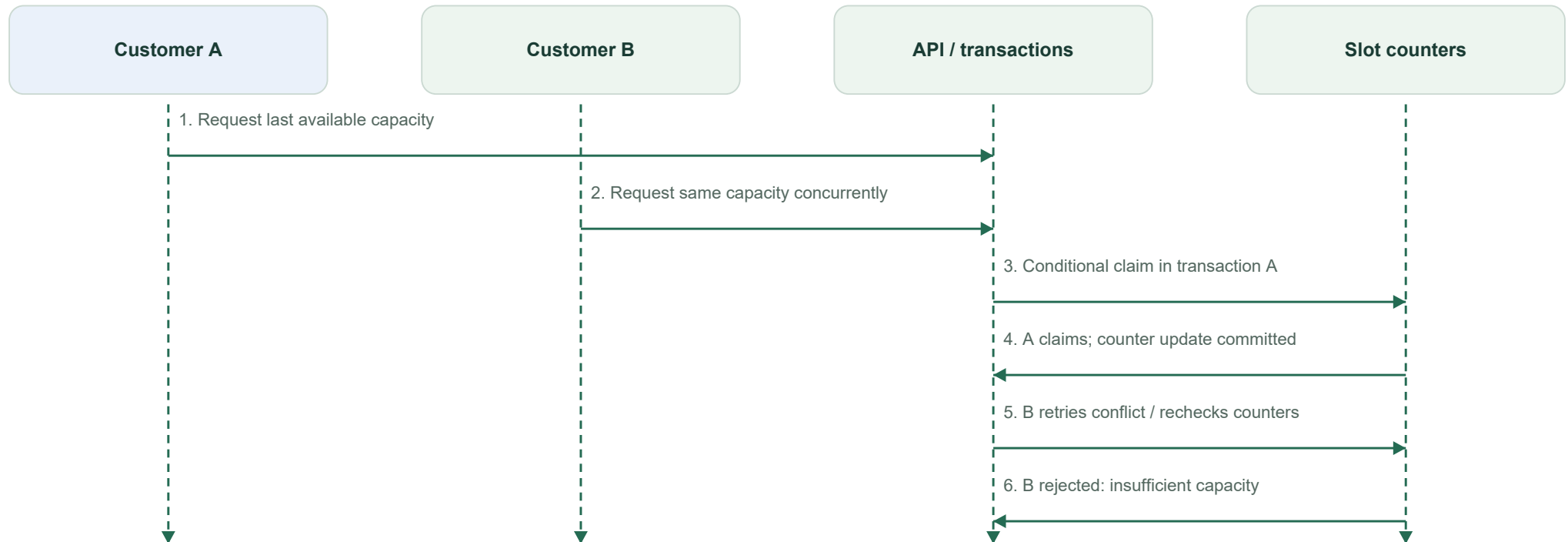
MongoStore.Transaction uses WithTransactionAsync with snapshot reads and majority writes.

Create reservation	Modify reservation
1. Choose target: actor for Prosumer; supplied prosumerId for staff. Reject missing target.	1. Load booking; enforce prosumer ownership. Require Pending/Approved state and old-start 12h notice.
2. Begin transaction; require target to exist as an Active Prosumer.	2. Require original prosumer active; decrement old slot holds inside the transaction.
3. Claim: read active slot; touch active station revision; atomically require ReservedCount < MaxBookings and ReservedKwh <= CapacityKwh - requested energy.	3. Claim selected new slot using the same conditional update. New start must be within 7 days and satisfy 12h notice.
4. Increment ReservedCount by 1 and ReservedKwh by requested energy. Validate future start within 7 days; copy slot start/end into reservation.	4. Replace slot/station/time/energy/direction; set Pending; clear QrNonce; increment Revision and UpdatedAt.
5. Insert Pending reservation and Reservation:Create audit; commit. Return joined BookingView.	5. Save booking + Reservation:Update audit; commit. Any failure rolls back release and claim together.

Cancelled/rejected/completed/expired reservations decrement the original held count and reserved energy. A failure after an attempted capacity update rolls back the entire transaction. Clients do not implement a separate capacity check as authority.

Concurrency and reservation invariants

A conditional write prevents oversubscription even when multiple clients see the same displayed availability.



Invariants: held count cannot exceed MaxBookings; held kWh cannot exceed CapacityKwh. Pending and Approved both hold capacity. Changes across booking/counter/audit commit atomically. Node revision writes also conflict with simultaneous deactivation and slot changes.

This is a transaction-level guarantee, not HTTP request deduplication. No client idempotency key is implemented; repeated creation requests can create separate bookings if capacity allows.

QR payload and completion algorithm

A QR is a signed reference to live state, not a self-contained authorization ticket.

Payload format: **v1.<reservationId>.<nonce>.<signature>**

Stage	Implementation
Approve / issue	Pending before start; require active prosumer; generate 16 random bytes as 32 hex characters. Increment reservation revision. QR endpoint signs v1.id.nonce with a dedicated HMAC-SHA256 key.
Signature verification	Require four components: v1; 32-character booking ID; 32-character nonce; 64-character signature. Decode signature and compare HMAC in constant time.
Live state validation	Booking exists; status Approved; nonce equals the QR nonce; now is not after end. For completion, now must also be at or after start.
Complete transaction	Recheck live QR; actual energy positive/finite and <= reserved. Release full hold; set Completed, CompletedBy, CompletedAt, TransferredKwh; clear nonce; increment revision; write audit.
Replay / stale QR	Changed, cancelled, rejected, expired or completed booking cannot use the previous QR to complete. Verify can run before start; Complete cannot.

The caller manually records actual kWh. Signature validity does not validate a meter reading, customer identity beyond the booking, or any payment.

Client implementation and validation details

These describe the latest workspace changes as well as existing behavior.

Component	Implementation detail
React Field / Modal	Shared number input: min=1 and step=1 for non-coordinate web fields; coordinates step=any. Modal turns numeric FormData into numbers and submits through src/api.js. Existing fractional API values are not rounded automatically.
Numeric API distinction	Energy/power DTOs remain double with positive ranges; counts remain int. Android/API may still accept fractional energy. The web whole-number choice does not change physical units or backend schema.
Android MainActivity	Status Spinner OnItemSelectedListener rebuilds the booking view when selected status changes. Initial selection is compared with current filter to avoid repeat reload. Search text retained; Search button handles explicit text search.
Stale UI responses	MainActivity generation increments when shell changes; callbacks from previous screen generations are ignored. Reservation pagination loads 20 rows per page.
Android MapActivity	GET /stations, no radius arguments; valid coordinates become GeoJSON features and a CircleLayer. Country bounds fit Sri Lanka plus returned station points; location update only changes the user indicator.
Android ApiClient	Single worker executor; HttpURLConnection; 15s connect/read timeouts; Bearer token; JSON request/response; UI-thread callback. Executor stopped on close.

Validation, errors and verification boundaries

Use these cases to explain or verify the design; proposed checks below are not claims that they were run for this PDF.

Case	Expected result
Past or >7-day booking start; <12h modification/cancel	Reject per server clock; exact 7-day and 12h boundaries accepted.
Maximum bookings exceeds node BatterySlots; slot overlap	Reject; changing dates/energy does not bypass battery count or overlap rules.
Concurrent claim of last capacity; node deactivation race	At most supported holds commit; transaction conflict retries and active-node touch protect consistency.
Prosumer reads or edits another booking; inactive session	Ownership denial; JWT rejected after deactivation/token-version change.
Tampered/old QR; early completion; excessive actual kWh	Reject signature/state/window/energy violations. Second completion fails live state check.
Map offline; status change; no location permission	Label cached map; status selection loads matching records; all active stations remain independent of user location.

Error families: 400 validation/rules; 401 unauthenticated or invalid token; 403 role/ownership/inactive-login denial; 404 missing entity; 409 capacity/state conflict; 429 auth rate limit; 503 Mongo failure; 500 unexpected error.

Repository validation entry points: backend/SolarTrading.Tests, npm run build in frontend/web, and Gradle assembleDebug lintDebug. No code changes or new test execution are implied by creating these guides.

Implementation references

These are repository paths, not external product documentation. The code is the source of truth for this guide.

backend/SolarTrading.Api/Models/Entities.cs and DTOs/Requests.cs

backend/SolarTrading.Api/Controllers/ApiControllers.cs and Repositories/MongoStore.cs

backend/SolarTrading.Api/Services/*.cs and Program.cs

frontend/web/src/main.jsx; Android MainActivity.java, MapActivity.java, LocalStore.java and ApiClient.java